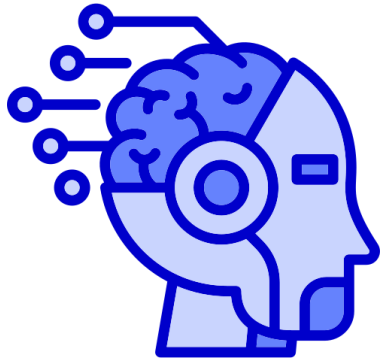
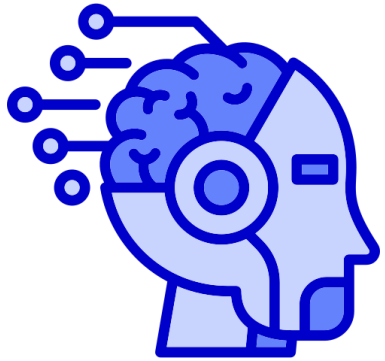


UnInformed Search

BFS, DFS, UCS

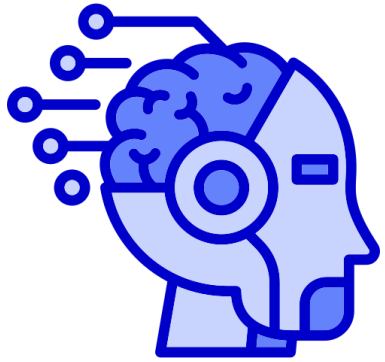
Artificial Intelligence

Outline

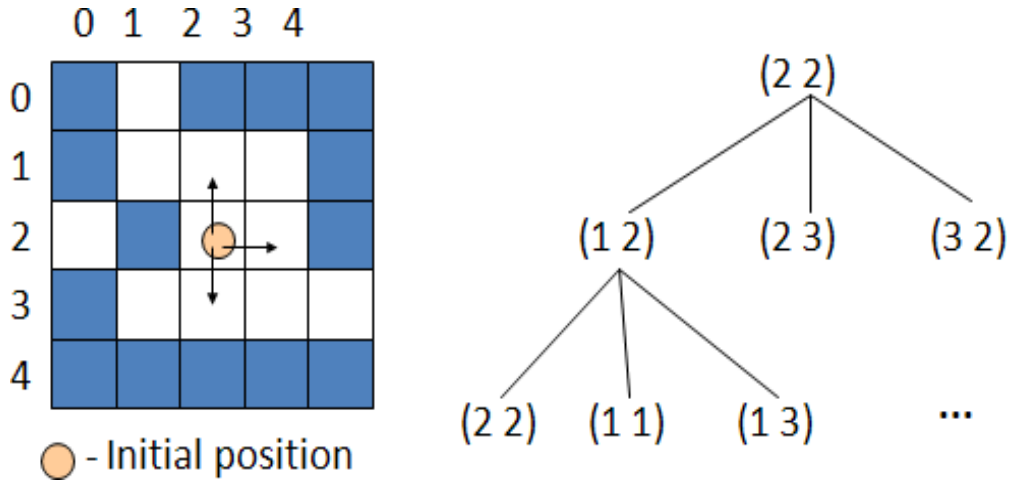


- Uninformed search
- Search strategies
- Problem formulation
- Uniform Search Strategies
  - Depth First Search
  - Breadth First Search
  - Uniform cost Search
  - Depth-Limit Search
  - Iterative Deepening Search
  - Bidirectional Search

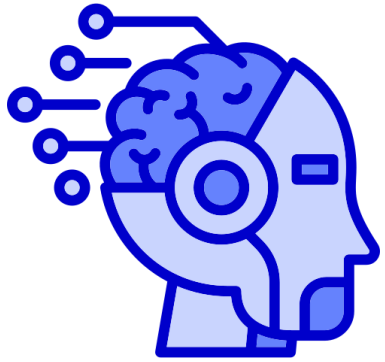
Uninformed Search Strategies



Uninformed (blind) strategies use only the information available in the problem definition



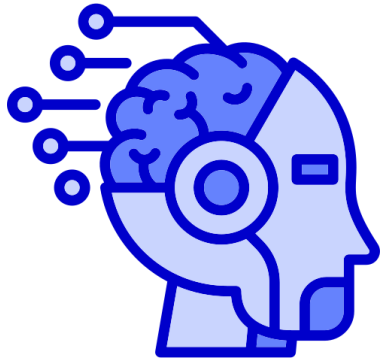
Uninformed Search Strategies



- An uninformed search algorithm is given no clue about how close a state is to the goal(s).

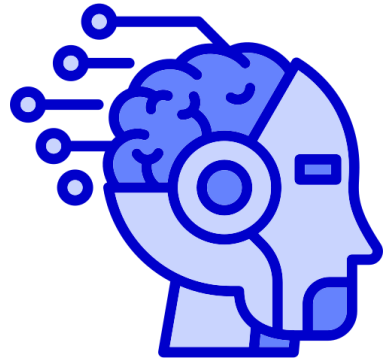
- For example, consider our agent in Arad with the goal of reaching Bucharest.
- An uninformed agent with no knowledge of Romanian geography has no clue whether going to Zerind or Sibiu is a better first step.

Breadth-First Search (BFS)

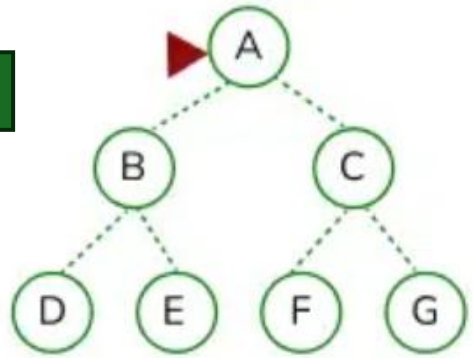


- Root node is expanded first -> successor of root node -> their successors
- All nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded
- Easy to implement by using **FIFO** queue

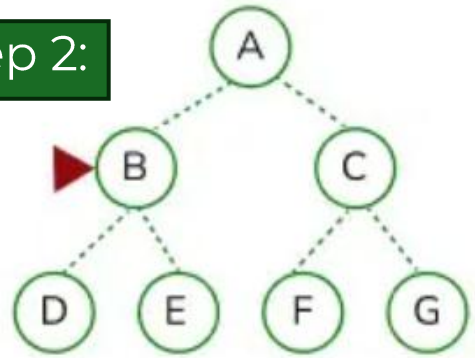
Breadth-First Search (BFS)



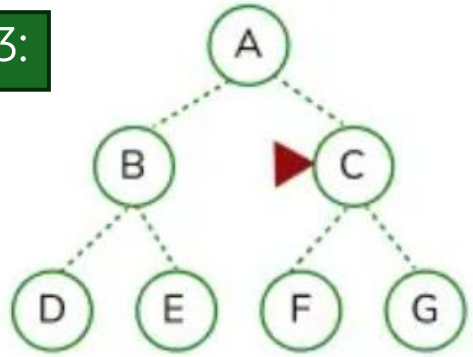
Step 1:



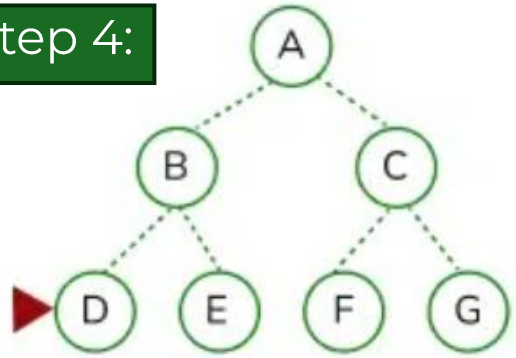
Step 2:



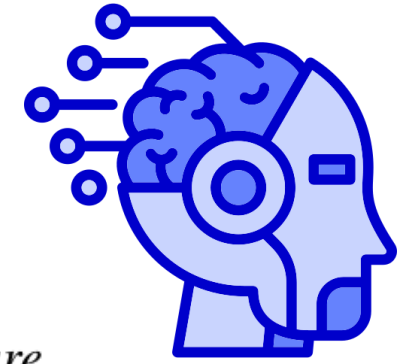
Step 3:



Step 4:

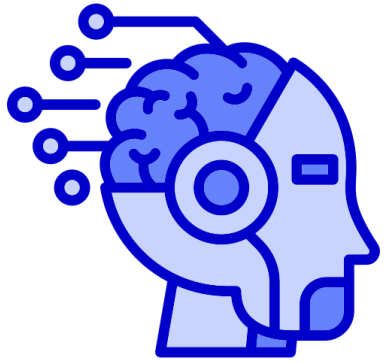


Breadth-First Search (BFS)



```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

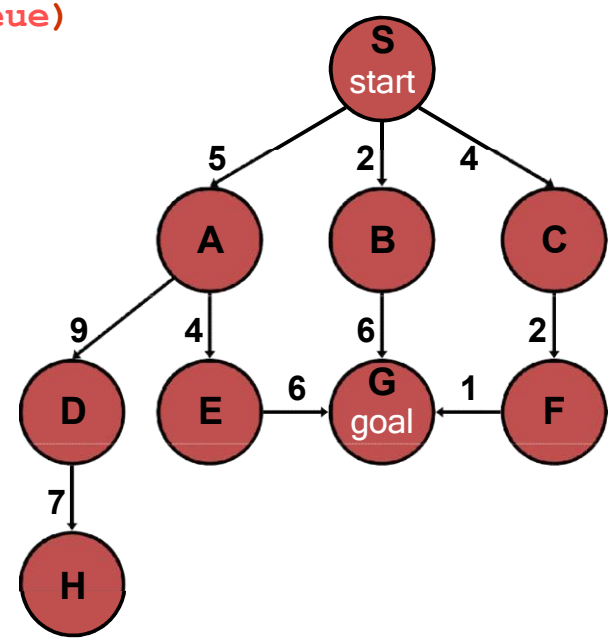
Breadth-First Search (BFS)



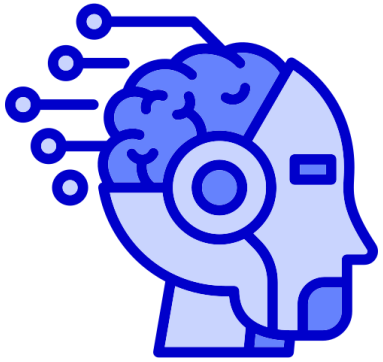
`generalSearch(problem, queue)`

# of nodes tested: 0, expanded: 0

expnd. node	Frontier list
	{S}



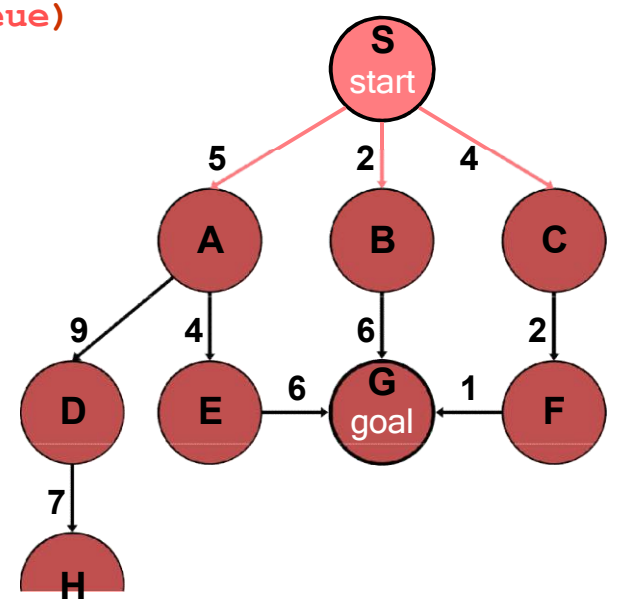
Breadth-First Search (BFS)



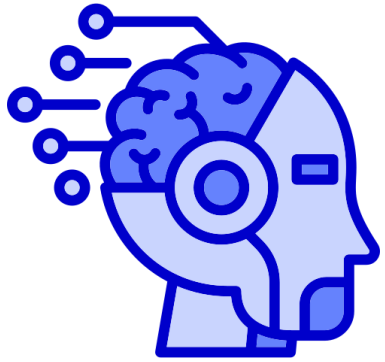
`generalSearch(problem, queue)`

# of nodes tested: 1, expanded: 1

expnd. node	Frontier list
	{S}
S not goal	{A,B,C}



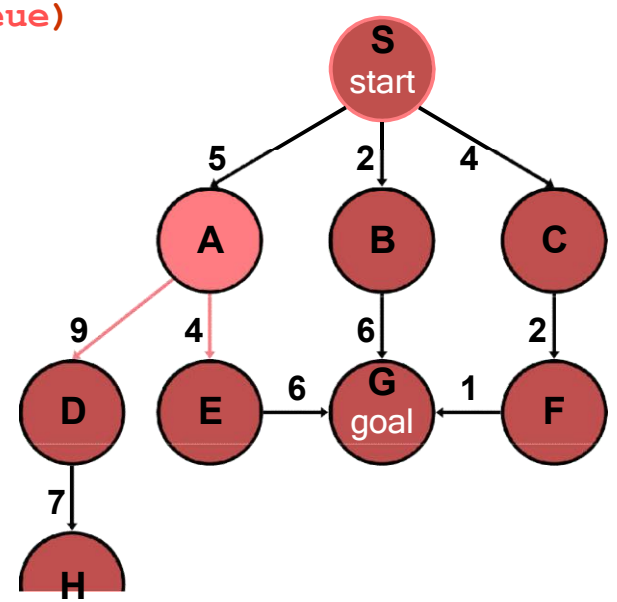
Breadth-First Search (BFS)

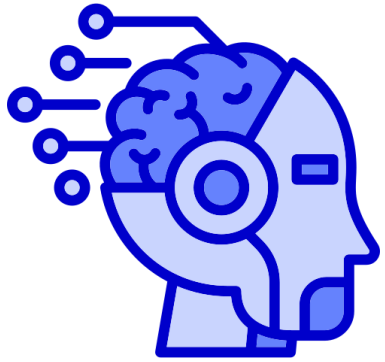


`generalSearch(problem, queue)`

# of nodes tested: 2, expanded: 2

expnd. node	Frontier list
	{S}
S	{A,B,C}
A not goal	{B,C,D,E}



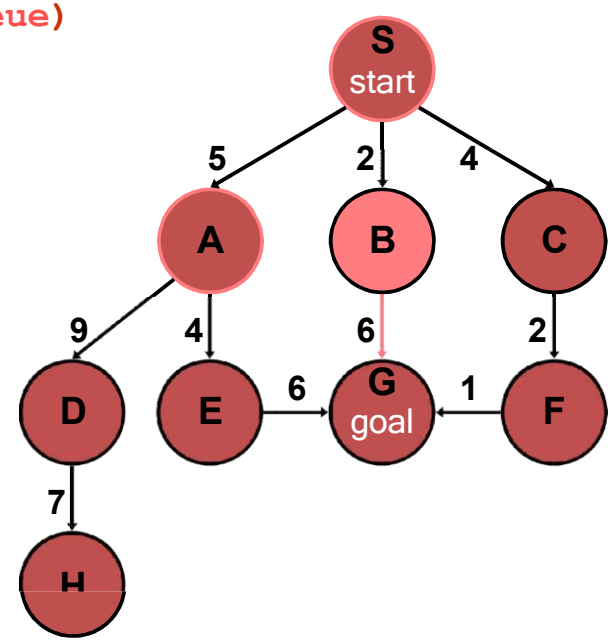


Breadth-First Search (BFS)

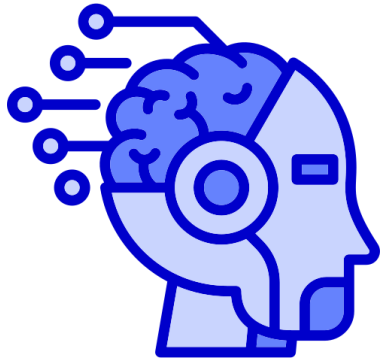
`generalSearch(problem, queue)`

# of nodes tested: 3, expanded: 3

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B not goal	{C,D,E,G}



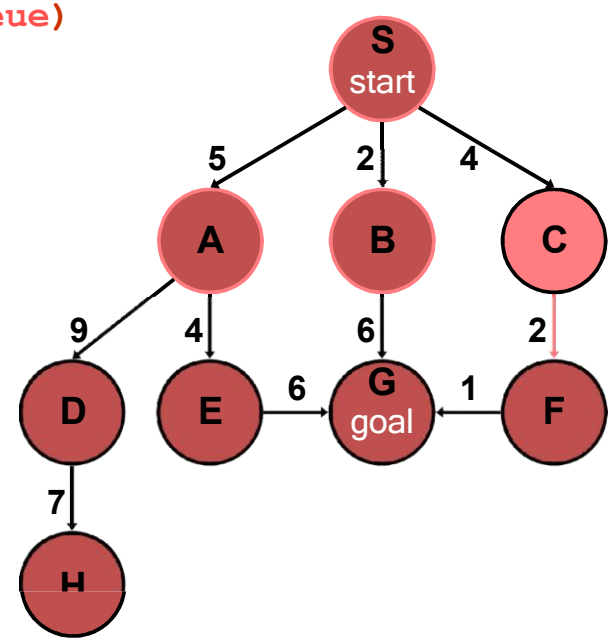
Breadth-First Search (BFS)

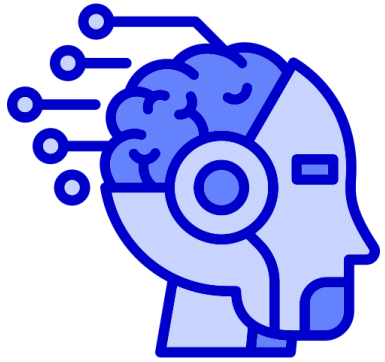


`generalSearch(problem, queue)`

# of nodes tested: 4, expanded: 4

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C not goal	{D,E,G,F}



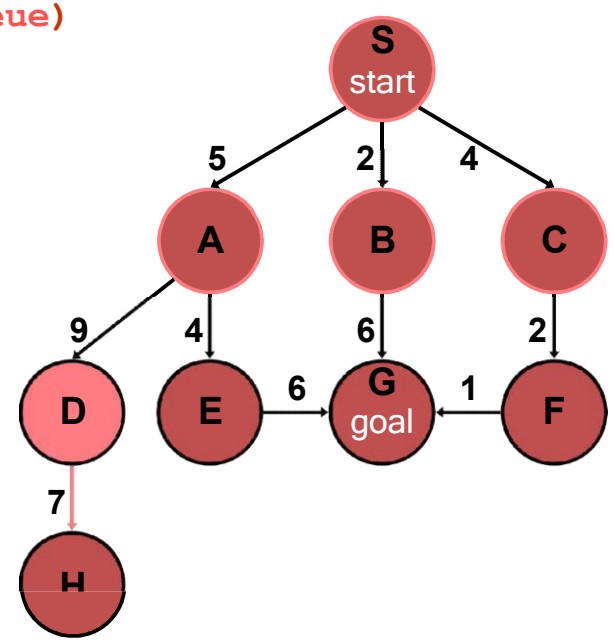


Breadth-First Search (BFS)

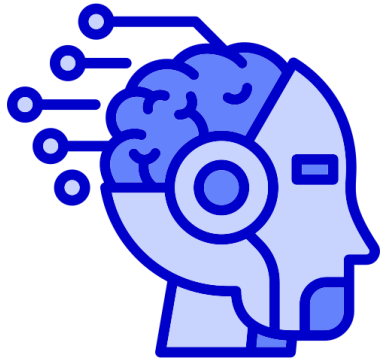
`generalSearch(problem, queue)`

# of nodes tested: 5, expanded: 5

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B.	{C,D,E,G}
C.	{D,E,G,F}
D not goal	{E,G,F,H}



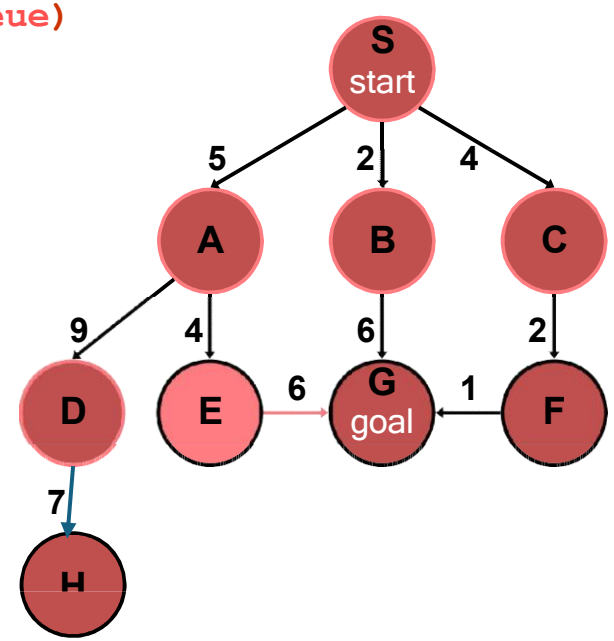
Breadth-First Search (BFS)

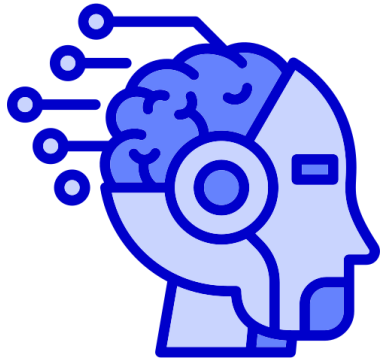


`generalSearch(problem, queue)`

# of nodes tested: 6, expanded: 6

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B.	{C,D,E,G}
C.	{D,E,G,F}
D	{E,G,F,H}
E not goal	{G,F,H,G}



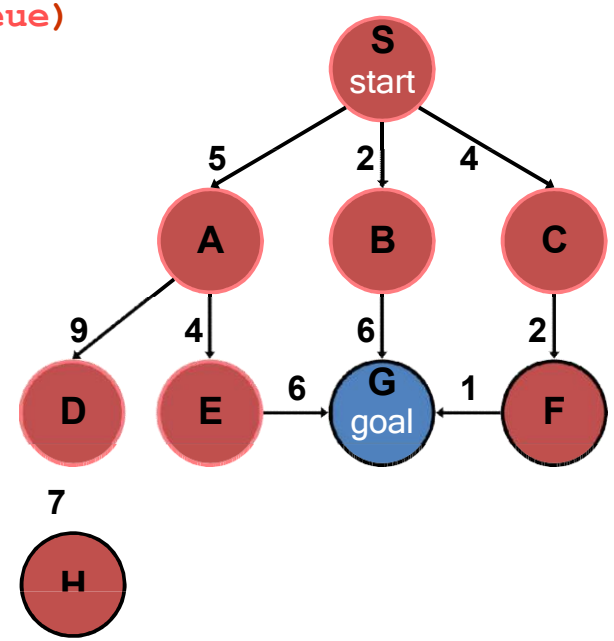


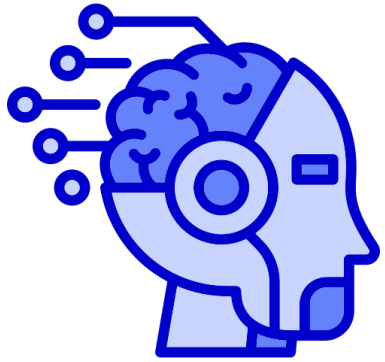
Breadth-First Search (BFS)

`generalSearch(problem, queue)`

# of nodes tested: 7, expanded: 6

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B.	{C,D,E,G}
C.	{D,E,G,F}
D.	{E,G,F,H}
E.	{G,F,H,G}
G goal	{F,H,G} no expand



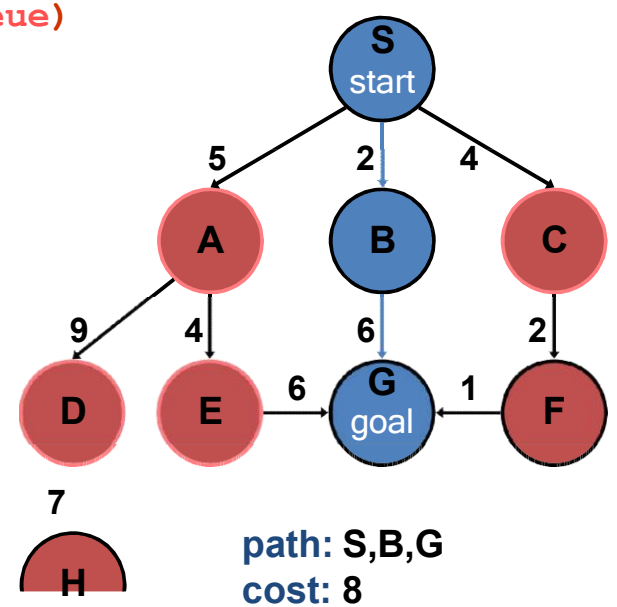


Breadth-First Search (BFS)

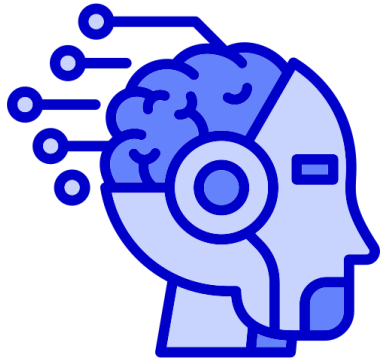
`generalSearch(problem, queue)`

# of nodes tested: 7, expanded: 6

expnd. node	Frontier list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B.	{C,D,E,G}
C.	{D,E,G,F}
D.	{E,G,F,H}
E.	{G,F,H,G}
G	{F,H,G}



Properties of Breadth-FirstSearch



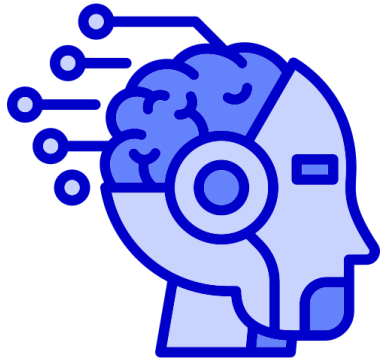
- Completeness: Yes, if *b* is finite
- Optimality: Yes (assuming cost = 1per step)
- Time complexity: $1+b+b^2+\ldots+b^d = O(b^d)$, i.e., exponential in *d*

- Space complexity: $O(b^d)$ Every node generated must remain in memory so it will be same as time complexity

b – maximum branching factor of the search tree
d – depth of the least-costsolution
m – maximum depth of the state space (may be infinite)

Exponential Growth

Space is bigger problem then time.

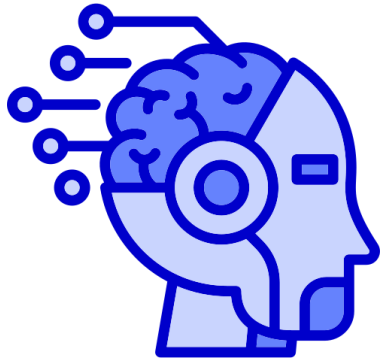


Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

Time and memory requirements for breadth-first search, assuming a branching factor of 10, 100 bytes per node and searching 1000 nodes/second

Using BFS



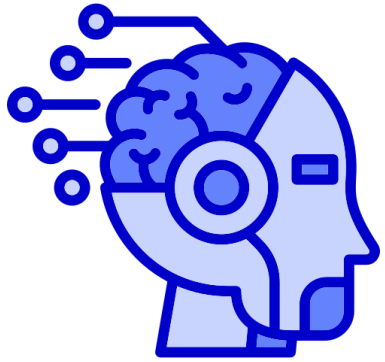
When is BFS appropriate?

- space is not a problem
- it's necessary to find the solution with the fewest arcs
- although all solutions may not be shallow, at least some are

When is BFS inappropriate?

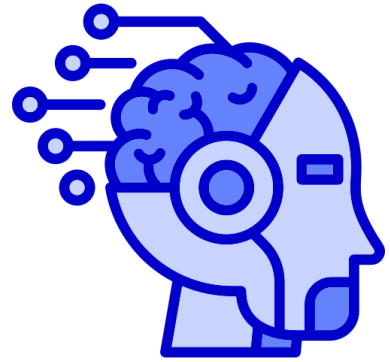
- space is limited
- all solutions tend to be located deep in the tree 11
- the branching factor is very large

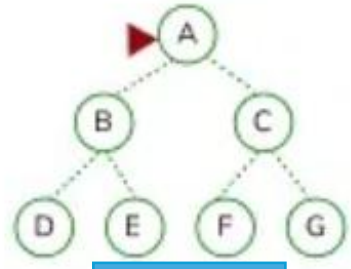
Depth First Search



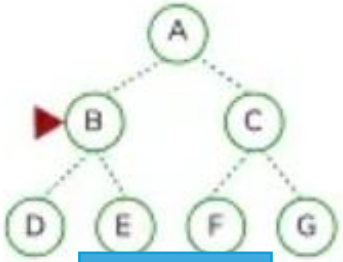
- Expands the **deepest node** in the current frontier of the search tree until the nodes have no successors, then backs up to the next deepest node that still has unexplored successors.
- **Use LIFO queue–** The most recently generated node is chosen for expansion, the deepest unexpanded node
- Use recursive

Depth First Search (DFS)

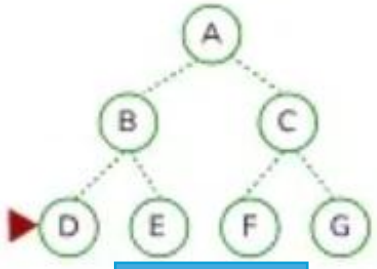




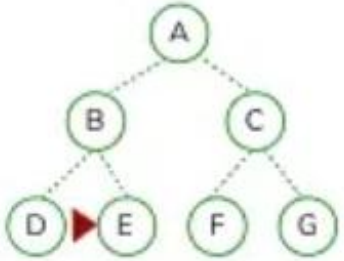
Step 1



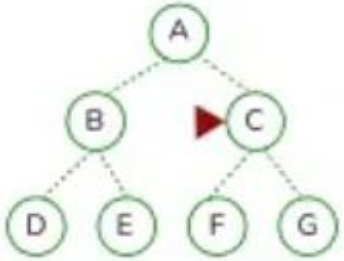
Step 2



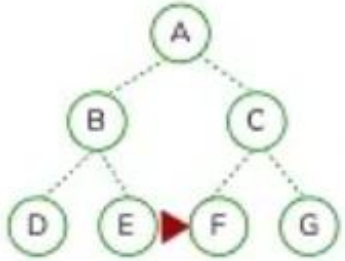
Step 3



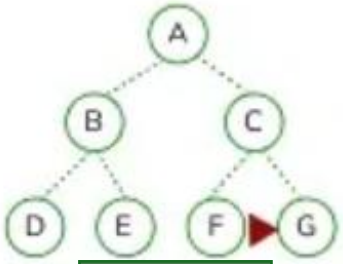
Step 4



Step 5



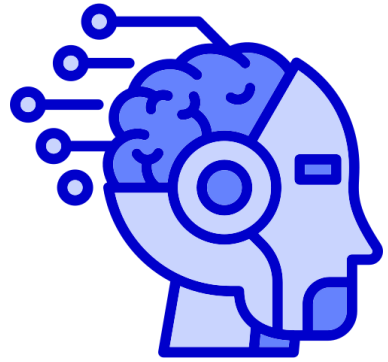
Step 6

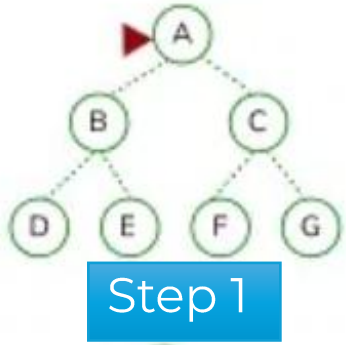


Step 7

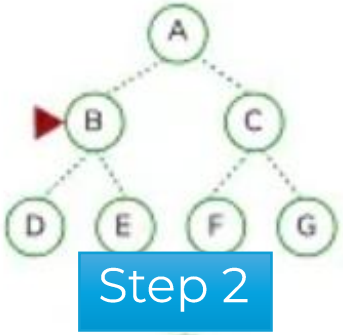
Artificial Intelligence

Depth First Search (DFS)

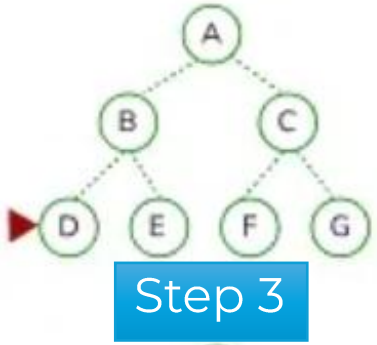




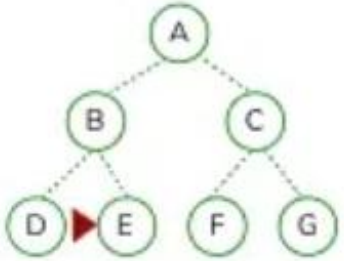
Step 1



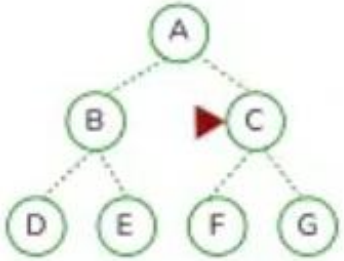
Step 2



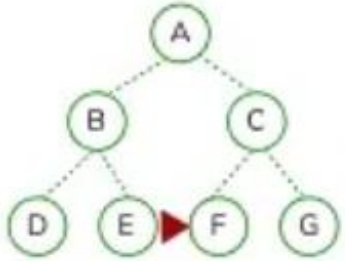
Step 3



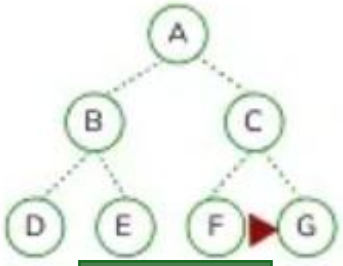
Step 4



Step 5



Step 6



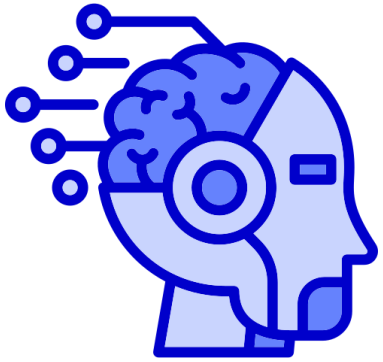
Step 7

A -> B -> D -> E -> C -> F -> G (Pre-Order)

D → B → E → A → F → C → G (In-Order)

D → E → B → F → G → C → A (Post-Order)

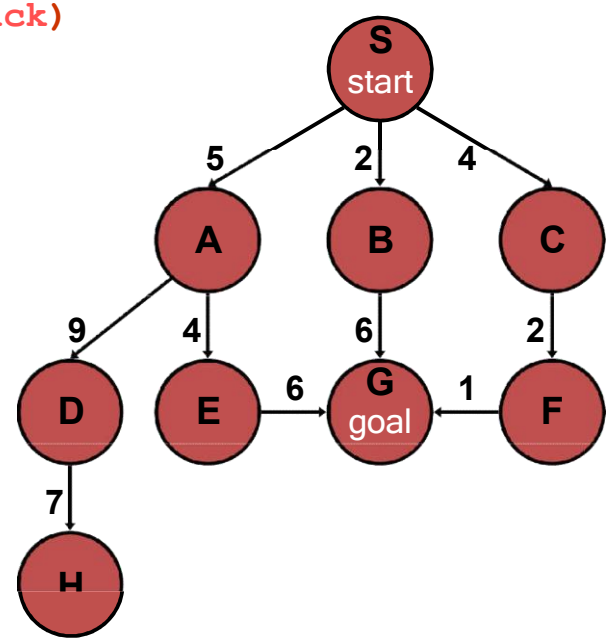
Depth-First Search (DFS)



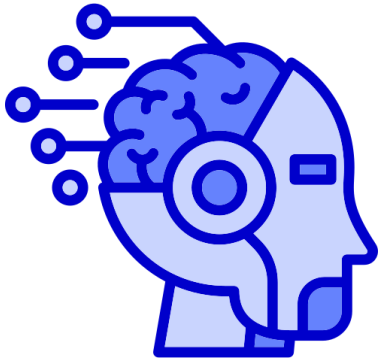
`generalSearch(problem, stack)`

# of nodes tested: 0, expanded: 0

expnd. node	Frontier
	{S}



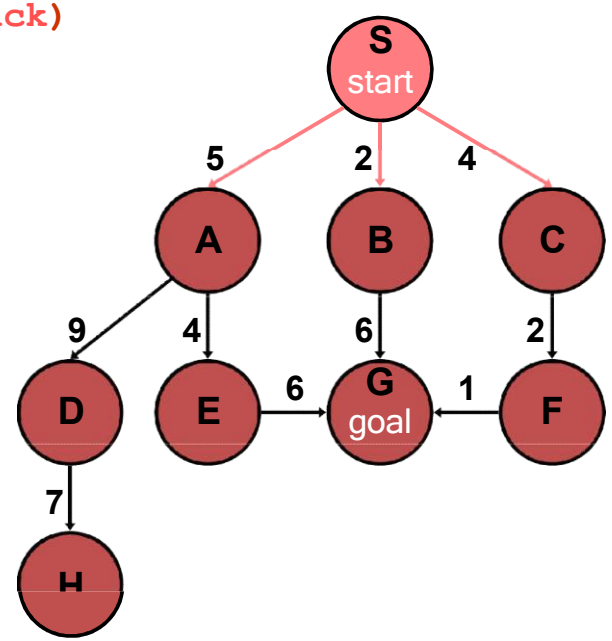
Depth-First Search (DFS)



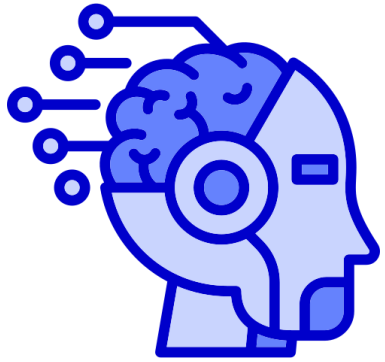
`generalSearch(problem, stack)`

# of nodes tested: 1, expanded: 1

expnd. node	Frontier
	{S}
S not goal	{A,B,C}



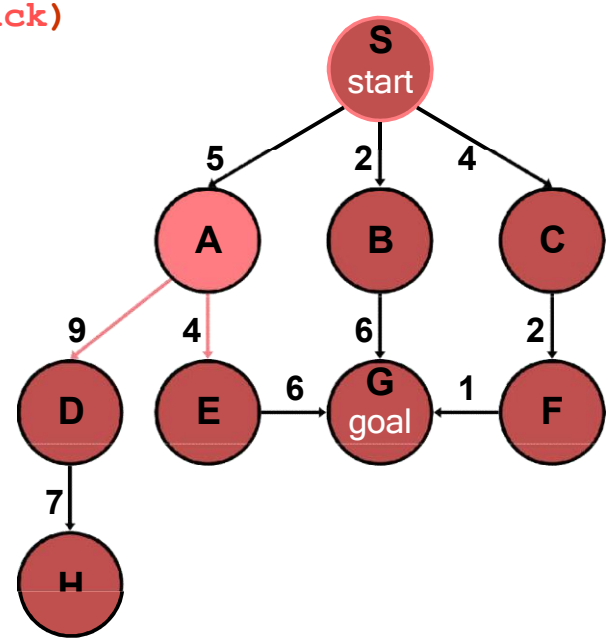
Depth-First Search (DFS)



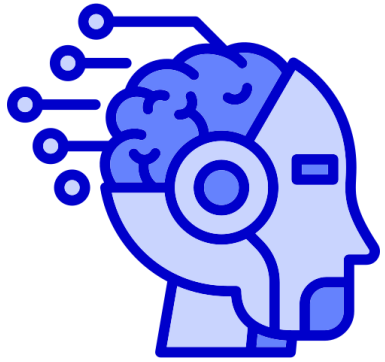
`generalSearch(problem, stack)`

# of nodes tested: 2, expanded: 2

expnd. node	Frontier
	{S}
S	{A,B,C}
A not goal	{D,E,B,C}



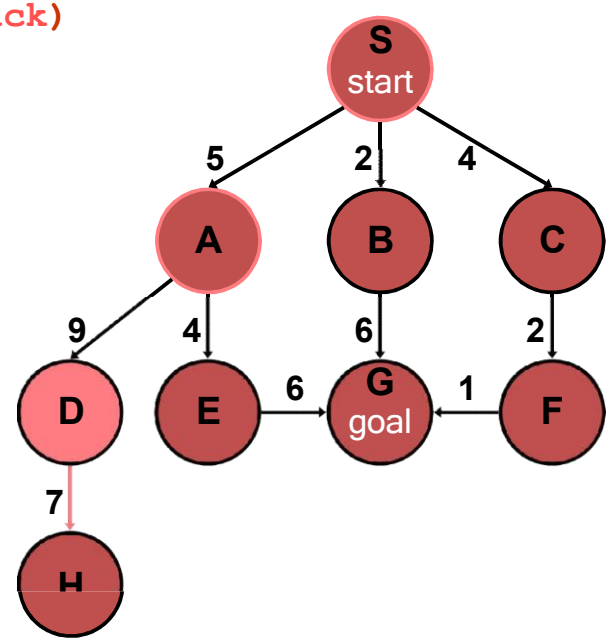
Depth-First Search (DFS)



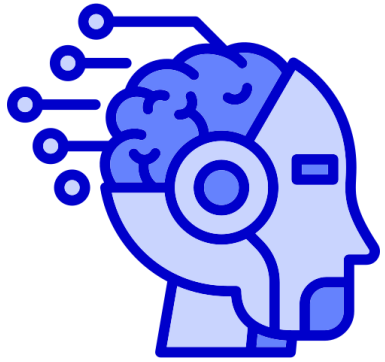
`generalSearch(problem, stack)`

# of nodes tested: 3, expanded: 3

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{D,E,B,C}
D not goal	{H,E,B,C}



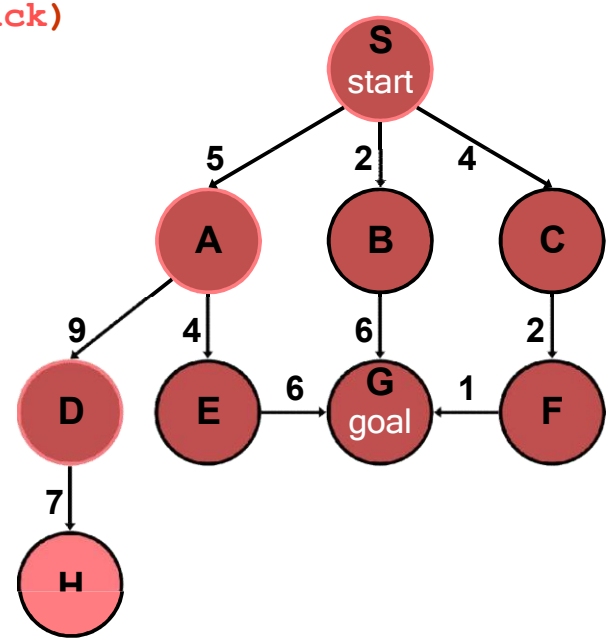
Depth-First Search (DFS)



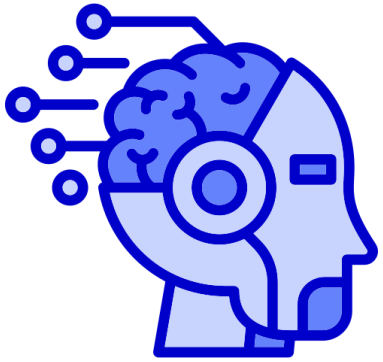
`generalSearch(problem, stack)`

# of nodes tested: 4, expanded: 4

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H not goal	{E,B,C}



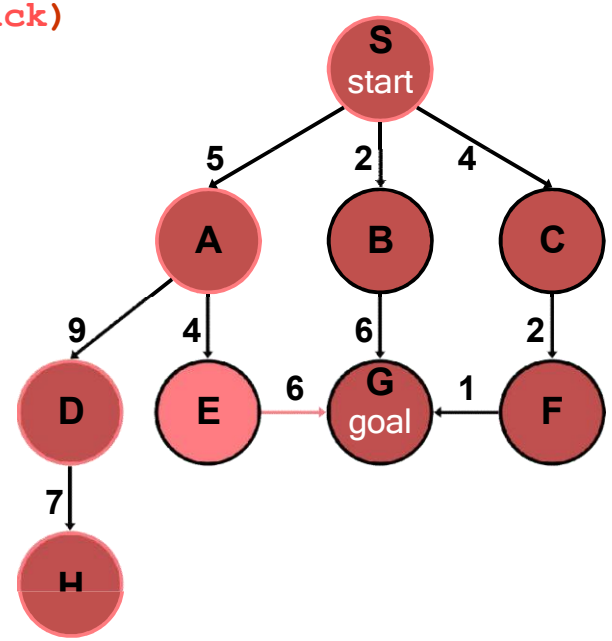
Depth-First Search (DFS)



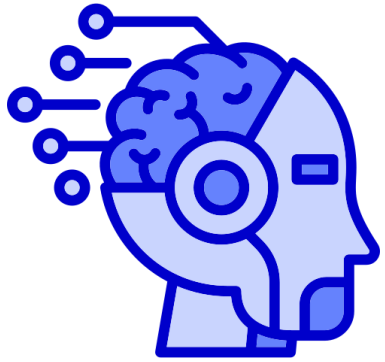
`generalSearch(problem, stack)`

# of nodes tested: 5, expanded: 5

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H	{E,B,C}
E not goal	{G,B,C}



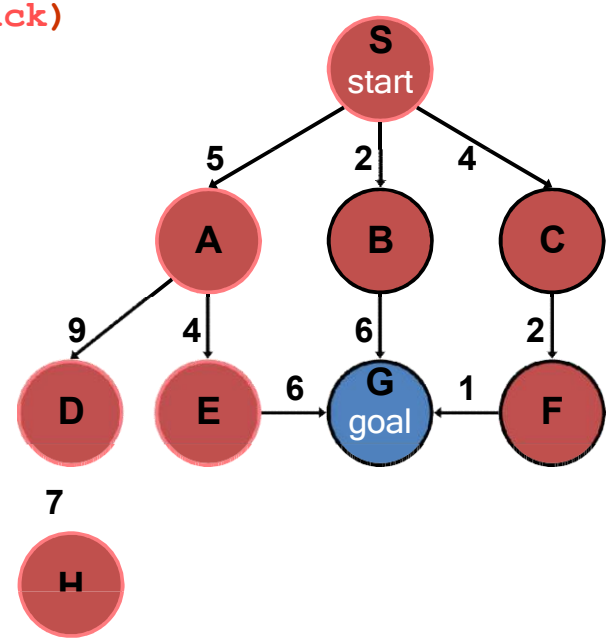
Depth-First Search (DFS)



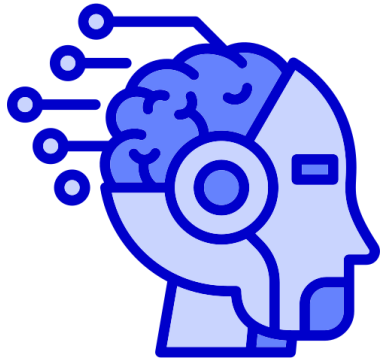
`generalSearch(problem, stack)`

# of nodes tested: 6, expanded: 5

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H	{E,B,C}
E	{G,B,C}
G goal	{B,C} no expand



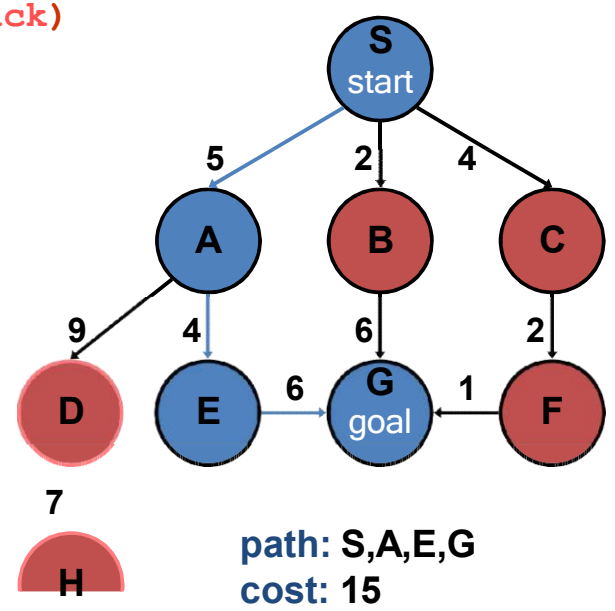
Depth-First Search (DFS)



`generalSearch(problem, stack)`

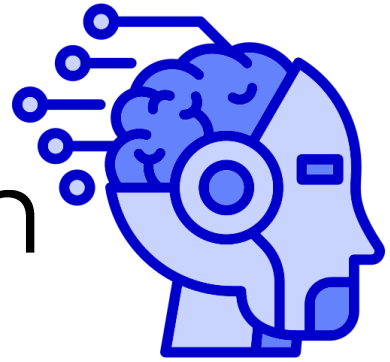
# of nodes tested: 6, expanded: 5

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H	{E,B,C}
E	{G,B,C}
G	{B,C}



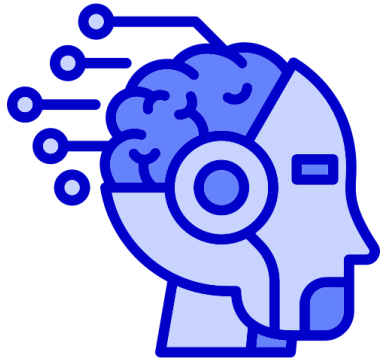
path: S,A,E,G
cost: 15

Properties of Depth-First Search



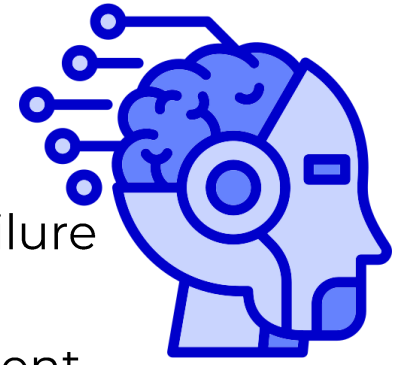
DFS Graph version is complete, and Tree version is incomplete. why?
Time Complexity: O(b^m)
If m (maximal depth) is much larger than d(depth of shallowest solution) – time is terrible ·
If there exist multiple solutions (in depth), DFS is faster than BFS.
Space Complexity: O(bd)

Uniform-Cost Search (UCS)



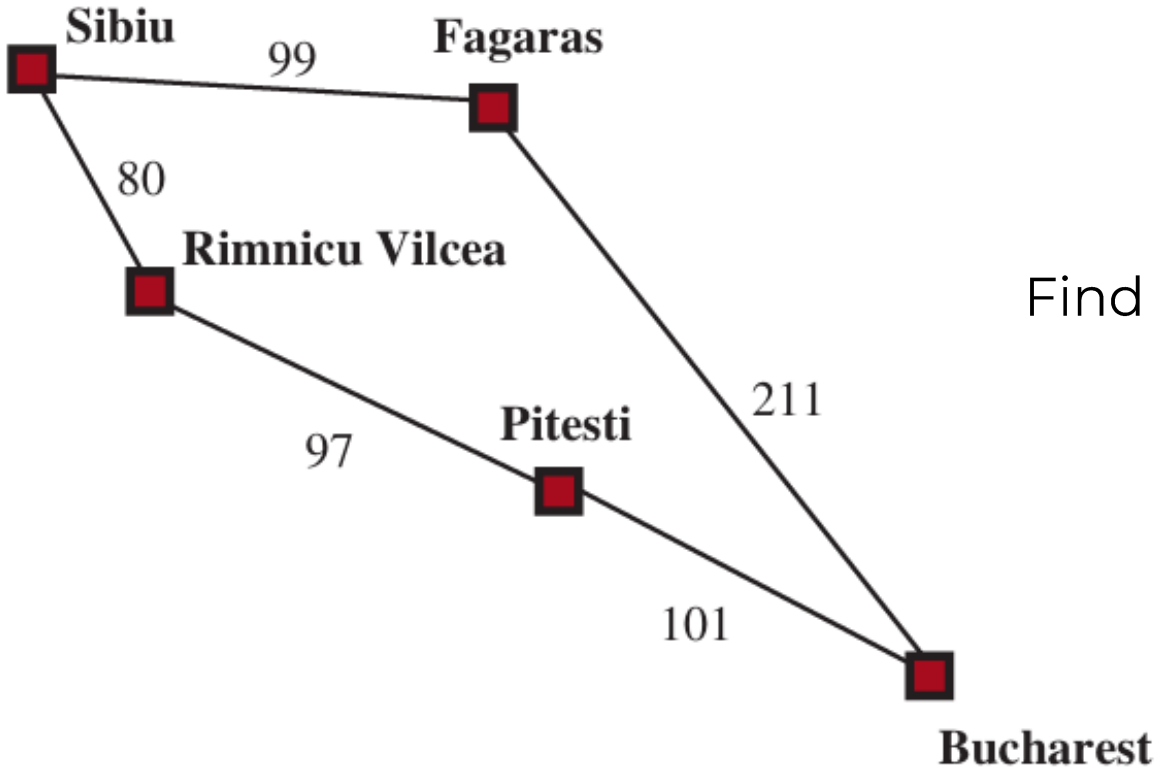
- Use a "Priority Queue" to order nodes on the *Frontier* list, sorted by path cost
- Let ***g*(*n*) = cost of the path from start node *s* to current node *n***
- Sort nodes by increasing the value of *g*

Uniform-Cost Search (UCS)



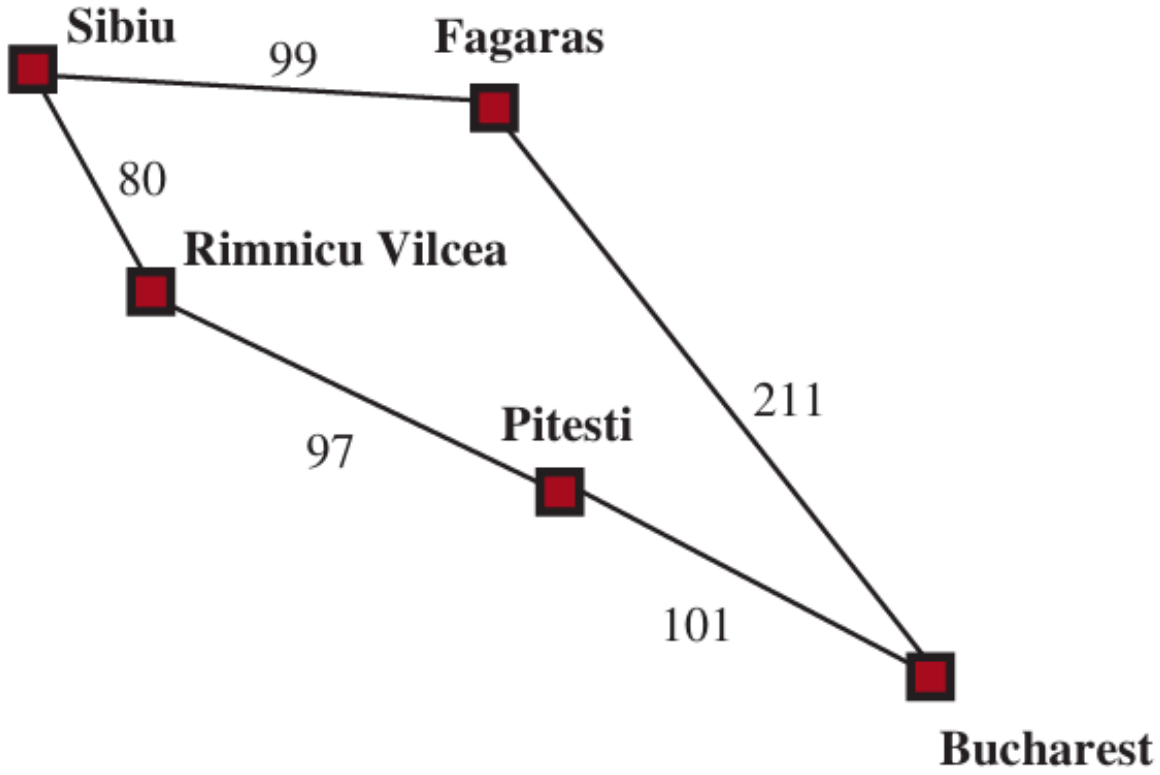
```
function UNIFORM-COST-SEARCH(problem) returns a solution node or failure
    node ← NODE(problem.INITIAL)
    frontier ← a priority queue ordered by PATH-COST, with node as an element
    reached ← empty map   // state → lowest path cost
    reached[node.STATE] ← 0
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)    // node with lowest path cost
        if problem.IS-GOAL(node.STATE) then
            return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            cost ← child.PATH-COST
            if s not in reached OR cost < reached[s] then
                reached[s] ← cost
                INSERT(child, frontier)
    return failure
```

Uniform-Cost Search (UCS)



Find path between Sibiu and Bucharest?

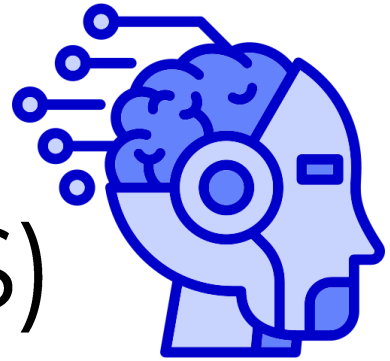
Uniform-Cost Search (UCS)



Step 1:
Sibiu->Fagaras or Sibiu -> Rimnicu?

Step 2:
Sibiu->Fagaras-> Bucharest
or
Sibiu -> Rimnicu -> Pitesti?

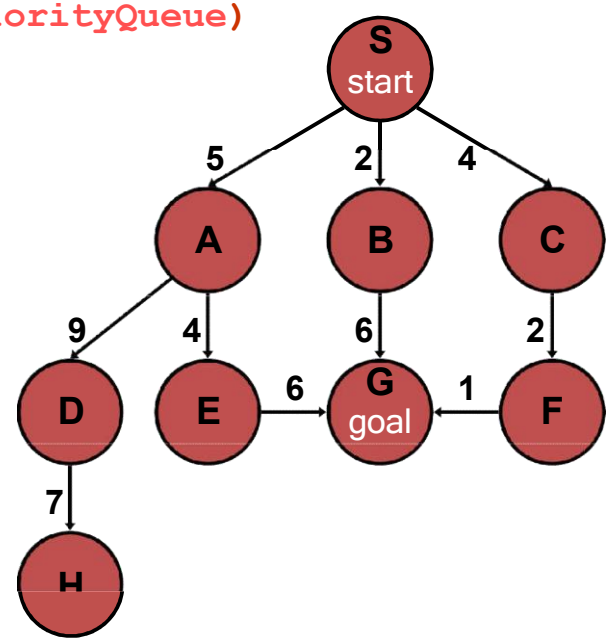
Uniform-Cost Search (UCS)



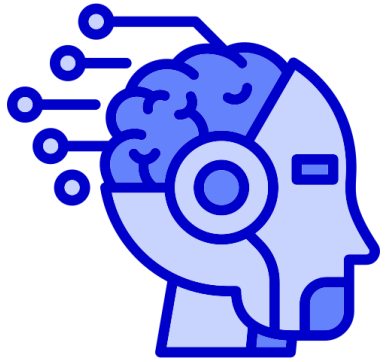
`generalSearch(problem, priorityQueue)`

# of nodes tested: 0, expanded: 0

expnd. node	Frontier list
	{S}



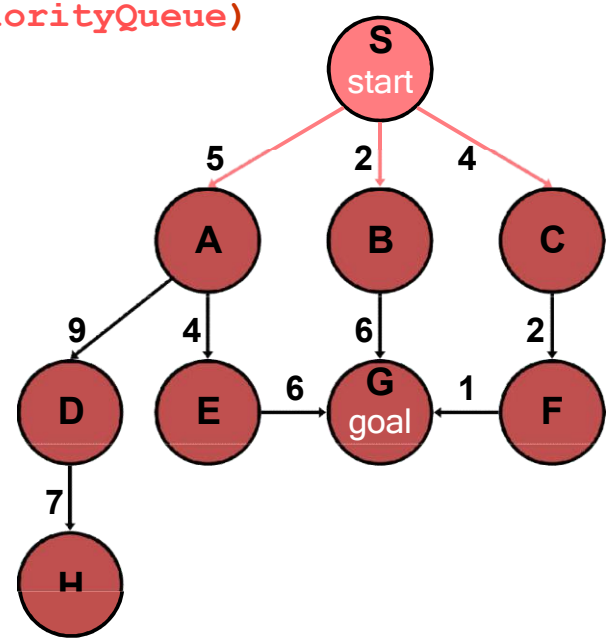
Uniform-Cost Search (UCS)



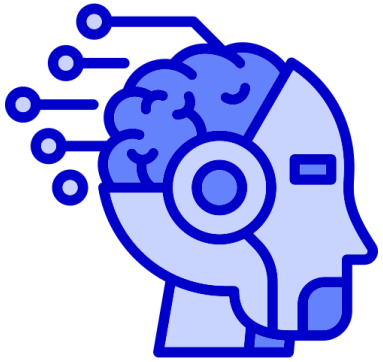
`generalSearch(problem, priorityQueue)`

# of nodes tested: 1, expanded: 1

expnd. node	Frontier list
	{S:0}
S not goal	{B:2,C:4,A:5}



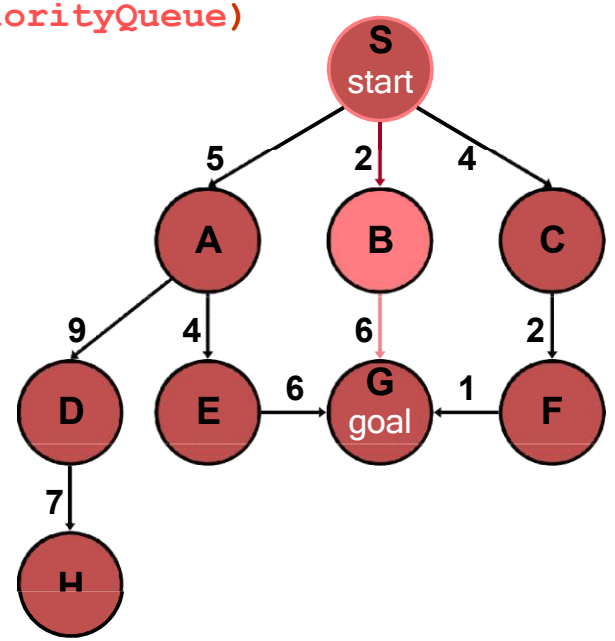
Uniform-Cost Search (UCS)



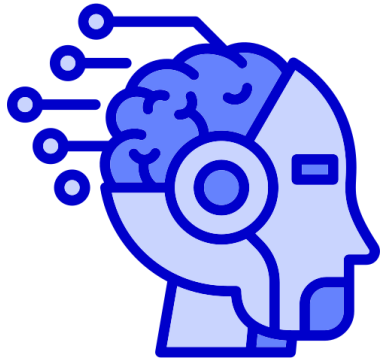
`generalSearch(problem, priorityQueue)`

# of nodes tested: 2, expanded: 2

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B not goal	{C:4,A:5,G:2+6}



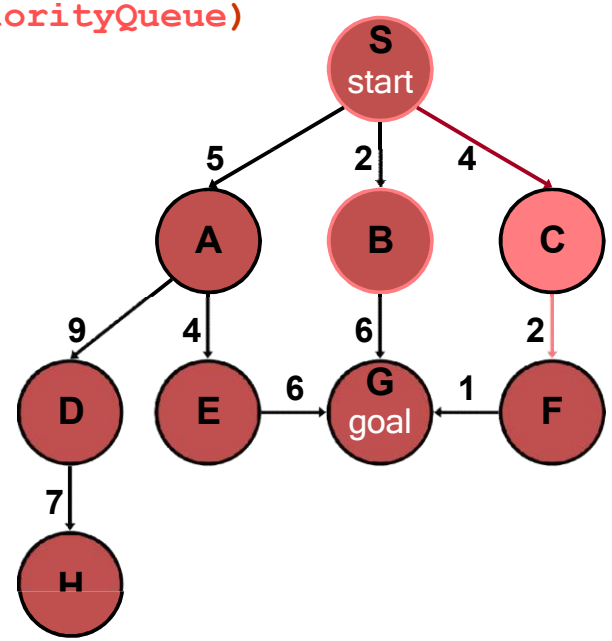
Uniform-Cost Search (UCS)

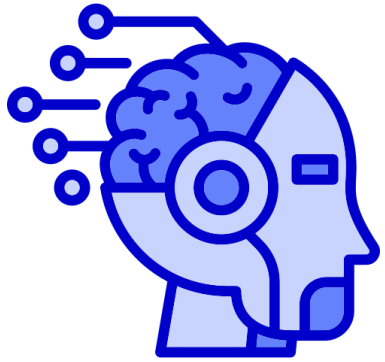


`generalSearch(problem, priorityQueue)`

# of nodes tested: 3, expanded: 3

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C not goal	{A:5,F:4+2,G:8}



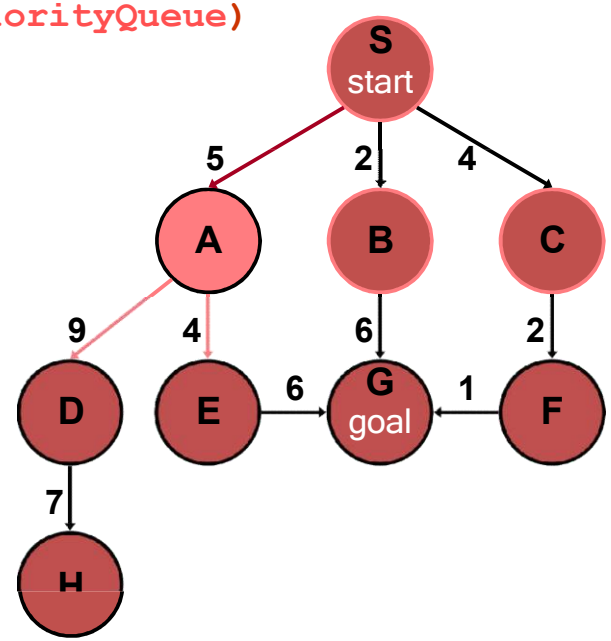


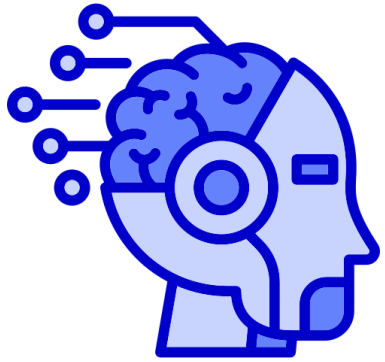
Uniform-Cost Search (UCS)

`generalSearch(problem, priorityQueue)`

# of nodes tested: 4, expanded: 4

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A not goal	{F:6,G:8,E:5+4 , D:5+9}



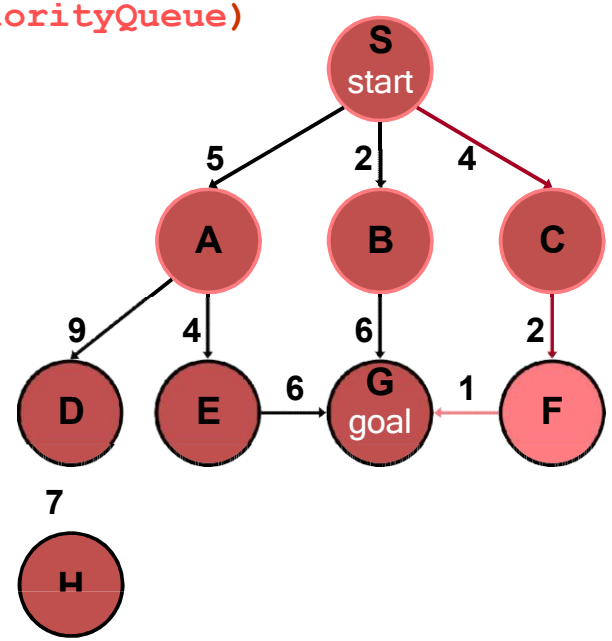


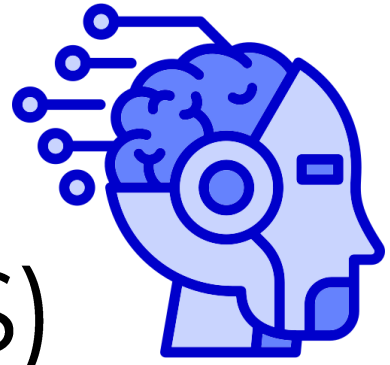
Uniform-Cost Search (UCS)

`generalSearch(problem, priorityQueue)`

# of nodes tested: 5, expanded: 5

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F not goal	{G:4+2+1,G:8,E:9 , D:14}



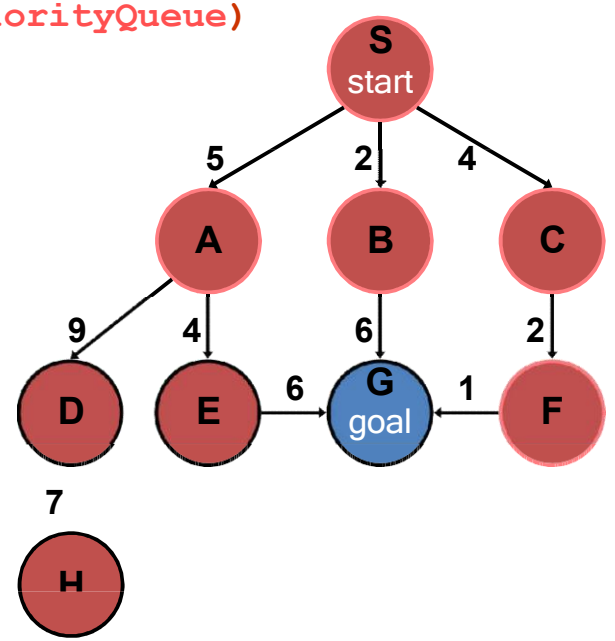


Uniform-Cost Search (UCS)

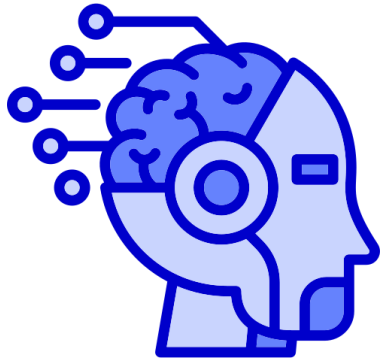
`generalSearch(problem, priorityQueue)`

# of nodes tested: 6, expanded: 5

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F	{G:7,G:8,E:9,D:14}
G goal	{G:8,E:9,D:14} no expand



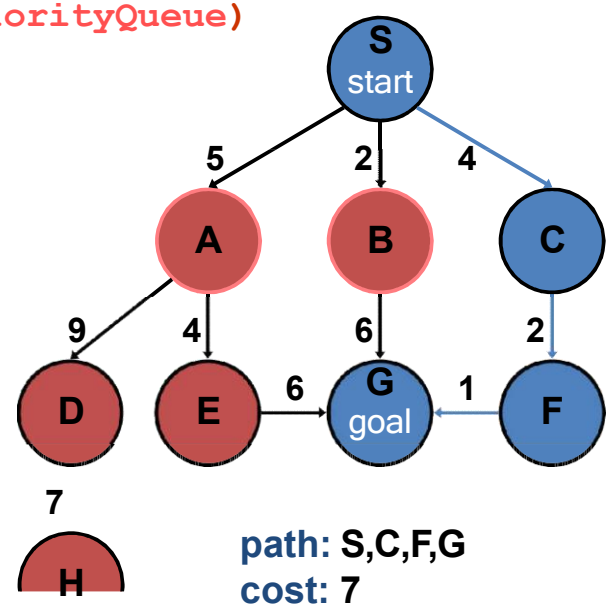
Uniform-Cost Search (UCS)



`generalSearch(problem, priorityQueue)`

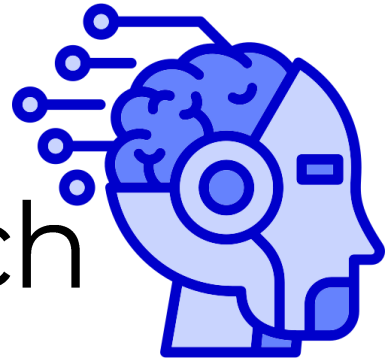
# of nodes tested: 6, expanded: 5

expnd. node	Frontier list
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F	{G:7,G:8,E:9,D:14}
G	{G:8,E:9,D:14}



path: S,C,F,G
cost: 7

Properties of Uniform-Cost Search



- Complete: Yes if arc costs > 0.
- Optimal: Yes
- Time and space complexity: $O(b^d)$ (i.e., exponential)
  - d is the depth of thesolution
  - is the branching factor at each non-leafnode

- More precisely, time and space complexity is $O(b^{C^*/\varepsilon})$ where a l edge costs $\varepsilon \sum > 0$, and is the best goalpath cost

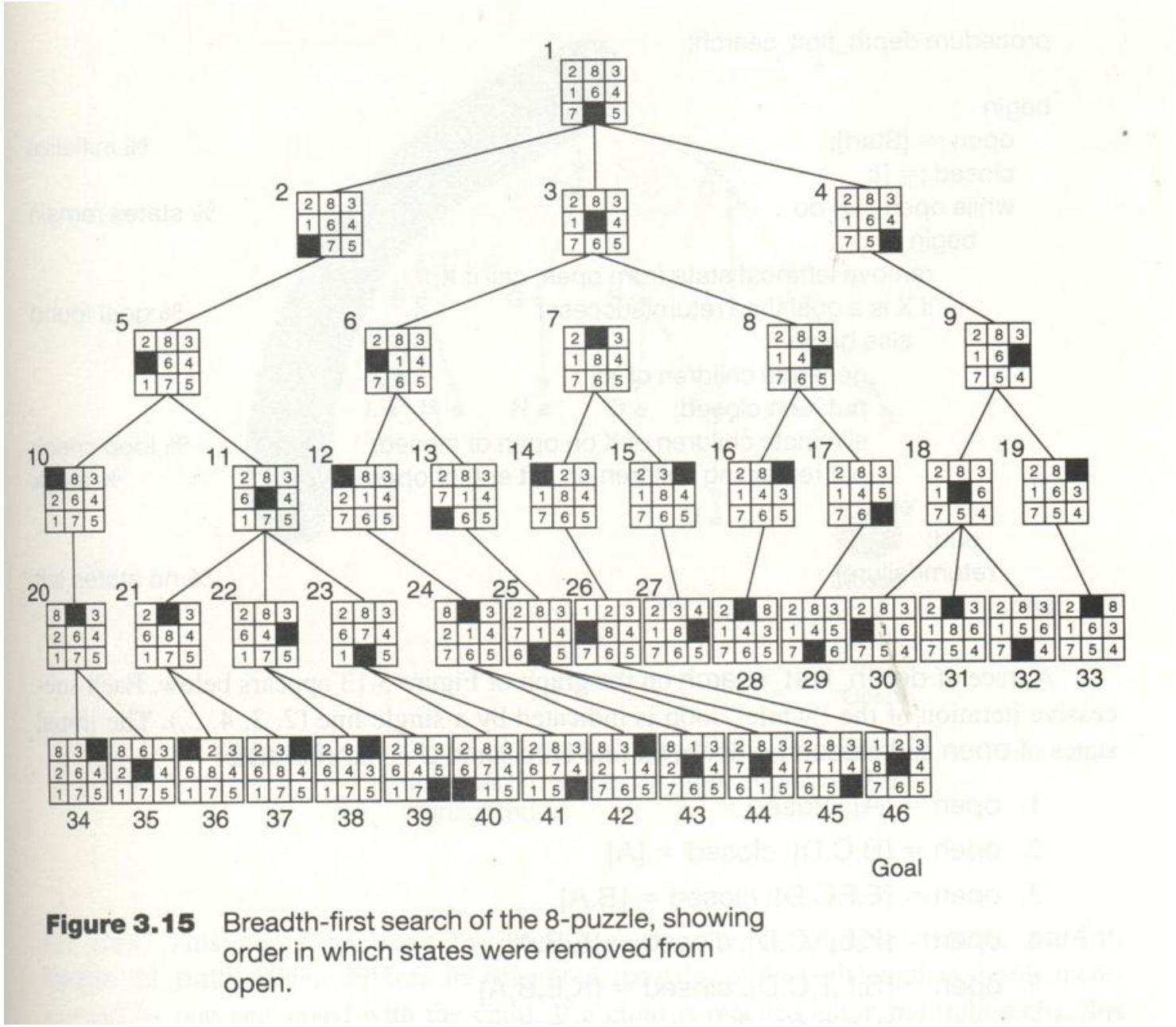
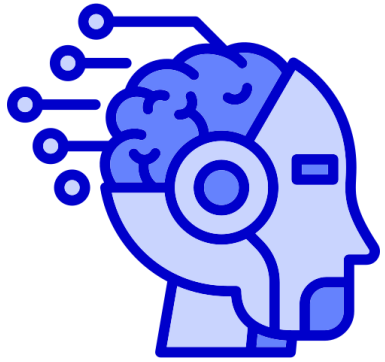


Figure 3.15 Breadth-first search of the 8-puzzle, showing order in which states were removed from open.



BFS on 8 Puzzle

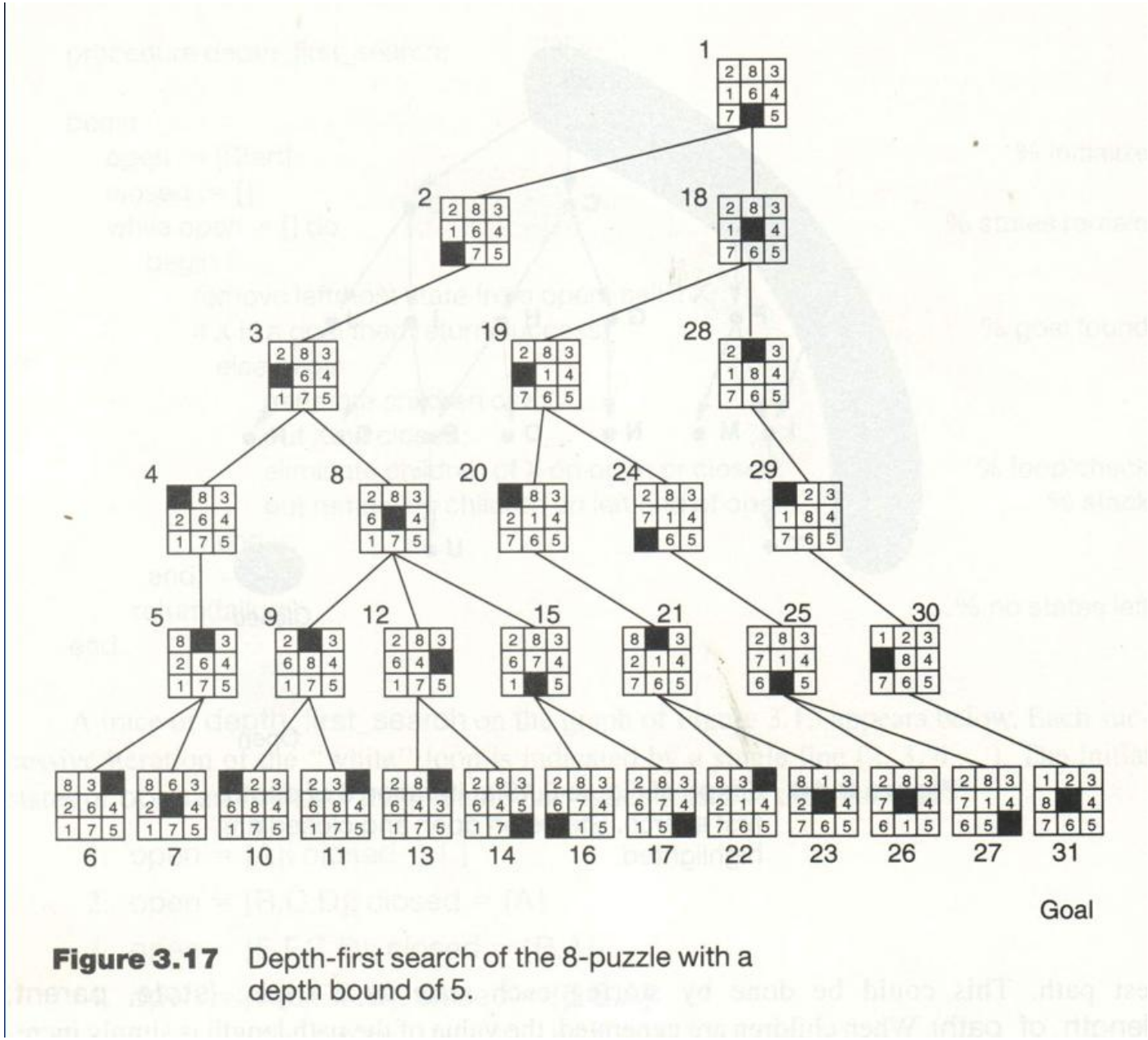
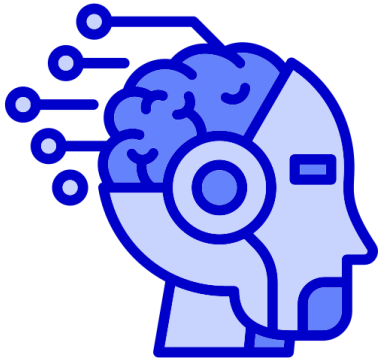


Figure 3.17 Depth-first search of the 8-puzzle with a depth bound of 5.

DFS on 8 Puzzle

HomeWork



Apply BFS, DFS , UCS , IDS on Pacman Game by defining actions, initial state, final state and path goal.